



UNIVERSITY OF
LIVERPOOL

COMP390

2024/25

Extending QMIX with Simple Techniques from Single-Agent Reinforcement Learning

Student Name: Lucheng Wang

Student ID: 201769571

Supervisor Name: Bei Peng

DEPARTMENT OF
COMPUTER SCIENCE

University of Liverpool
Liverpool L69 3BX

I dedicate this dissertation to all my efforts during my undergraduate journey, and to friends, teachers, and mentors who have supported and guided me along the way.

Acknowledgements

I would like to express my sincere gratitude to my mentor, Dr. Bei Peng, for providing me with the invaluable opportunity to work on projects related to reinforcement learning throughout the summer vacation and my final year at the University of Liverpool. Her support, guidance, and encouragement have greatly benefited me during the research process.

I am also deeply grateful to my parents, Yikan Wang and Jiaying Cheng, who have offered me unwavering support throughout my four years of undergraduate study. Thanks also to all the teachers and friends for their constant encouragement and companionship. Special thanks go to Jingyuan Sun for his generous help, which significantly contributed to my journey in studying deep learning.

This project is based on the PyMARL framework, developed by DeepMind, which is licensed under the Apache License 2.0.

A copy of the license can be found at <http://www.apache.org/licenses/LICENSE-2.0>.

Abstract

QMIX is a well-known and foundational Q-learning algorithm for cooperative multi-agent reinforcement learning (MARL) with a single joint reward signal. It operates under the centralized training and decentralized execution (CTDE) paradigm. QMIX combines the individual action-value functions into a centralized state-action value function. Each agent in QMIX maintains a local Q-network with the same architecture as in the deep Q-network (DQN). However, different from independent Q-learning (IQL), the parameters of networks are updated under the guidance of TD loss derived from Q-tot. Numerous improvements have been proposed to enhance the performance of DQN. Therefore, it is reasonable to assume that single-agent extensions have the potential to strengthen the performance of QMIX. This project concentrates on three techniques: Dueling Network, Multi-Step Learning, and Priority Replay Buffer. These three enhancements correspond to three different components of the model: the Dueling Network modifies the agent network architecture, Multi-Step Learning improves the TD target function, and the Priority Replay Buffer optimizes the sampling strategy. In addition, the project investigates the impact of environmental parameters on the effectiveness of the implemented approaches. Experiments have been conducted in multiple predefined predator-prey environments. Compared to the benchmark algorithm QMIX, Dueling Network and Multi-Step Learning contribute to some performance improvements, whereas Priority Replay Buffer introduces a negligible influence. Overall, this work leads to a further understanding of how specific single-agent methods affect MARL. By decomposing the architectural, algorithmic, and sampling aspects, this project provides a modular perspective for improving the performance and robustness of QMIX.

Statement of Ethical Compliance

This project involves data and participant categories A0. I guarantee that I have obeyed the ethical guidelines throughout the project.

Table of Contents

1	Introduction & Background.....	6
1.1	Introduction	6
1.2	Background	7
1.2.1	Deep Q-learning.....	7
1.2.2	QMIX	7
1.2.3	Rainbow	8
1.2.4	PyMARL.....	9
1.2.5	Predator-Prey.....	10
1.2.6	Barkla	10
2	Design.....	10
2.1	Design Overview	10
2.2	Dueling Network	10
2.3	Multi-step Learning.....	12
2.4	Priority Replay.....	13
2.5	Predator-Prey	14
2.6	Experimental Design	15
2.6.1	Comparative Experiments.....	16
2.6.2	Evaluation Metrics	16
3	Implementation	17
3.1	Overview of the Implementation	17
3.2	Implementation of Techniques.....	18
3.2.1	Dueling Network	18
3.2.2	Multi-step Learning.....	19
3.2.3	Priority Replay Buffer	19
3.3	Runtime Environment Configuration and Experiment Execution.....	20
3.4	Logging and Evaluation Tool	22
4	Testing & Evaluation	22
4.1	Algorithm Comparison Experiments.....	22
4.2	Environment Comparison Experiments	28
5	Project Ethics.....	30
6	Conclusion & Future Work	30
6.1	Conclusion	30
6.2	Future Work.....	30
7	BCS Project Criteria & Self-Reflection	31
8	References	32
9	Appendices.....	34

1 Introduction & Background

1.1 Introduction

Cooperative MARL has significant potential in solving a wide range of complex real-world problems, such as collision avoidance in navigation [1], search result diversification [2], autonomous cars [3], and robot swarm coordination [4]. However, two major challenges in MARL are scalability and partial observability. With an increasing number of agents, the joint state-action space grows exponentially, leading to suboptimal Nash equilibria, resource overuse and congestion, unpredictable and complex dynamics, and reinforcement of undesirable behaviors. Moreover, the enlarged state-action space can also result in high training complexity, low sample efficiency, and uncertainty in convergence during training [5]. Partial observability constrains that each agent has to decide upon incomplete and local information, its observation-action history, which raises the complexity of the learning task. Furthermore, the problem will aggravate under communication restrictions, where effective coordination becomes harder to achieve [6]. These challenges are formally captured in the Decentralized Partially Observable Markov Decision Processes (Dec-POMDPs) framework, which provides a general model for decentralized decision making in environments with uncertainty and incomplete information [7,8]. To address the difficulties inherent in Dec-POMDPs, the centralized training with decentralized execution (CTDE) paradigm was proposed as a state-of-the-art method and has attracted significant attention. The underlying concept of CTDE is that agents are allowed to access global information during training, which is referred to as centralized training. However, each agent has to act according to its history, known as decentralized execution [9]. Building on the CTDE paradigm, several algorithms have been proposed, including Multi-Agent Deep Deterministic Policy Gradient (MADDPG) [9], COMA [10], Value-Decomposition Networks (VDN) [11], and QMIX [12]. The Individual-Global-Max principle guarantees that the joint greedy action set is equivalent to the union of individual greedy actions, ensuring the policy consistency during centralized training and decentralized execution, and enabling scalable centralized training through the efficient calculation of the joint Q-function TD target [6]. QMIX is an algorithm developed based on earlier approaches like VDN. It employs the nonlinear mixing network to ensure the consistency of the joint Q-value function under the IGM principle and utilizes the hyper network to compute the parameters of the mixing network.

QMIX has indicated its promising performance among different tasks, such as the StarCraft Multi-Agent Challenge (SMAC), matrix game, and predator-prey. Despite its achievements, QMIX is limited by its monotonicity constraint, which prevents its ability to represent nonmonotonic joint action value functions [13] and undermines its performance in more challenging environments [14]. These highlighted the fact that this algorithm still has great potential for improvement and optimization. The architecture of QMIX can be structurally divided into two components: the individual Q-networks and the mixing network. For each agent, a Q-network with the same architecture as DQN is maintained to estimate its local Q-value. Given the structural similarity between individual Q-networks and DQN, and other reasons above, QMIX is selected as the base multi-agent algorithm in this project. In the Rainbow paper, various techniques are proposed to enhance the performance and stability of DQN, including Double Q-learning, Priority replay, Dueling networks, multi-step learning,

Distributional RL, and Noisy Nets [15], showing significant improvement. Thus, leveraging the techniques introduced in Rainbow to enhance QMIX is also a reasonable opinion. This project focuses on three enhancements: Dueling networks, Multi-step Learning, and Priority replay. Among them, the Dueling network modifies the agent Q-network architecture by splitting the action-value function into a state-value function and an advantage function. Multi-step learning extends the traditional temporal-difference (TD) methods by integrating multiple future rewards. Priority replay alters the sampling probability over transitions according to the TD error. The experiments of these techniques are conducted on multiple predator-prey environments. Additionally, the project investigates the relationship between algorithmic and environmental parameter changes and the performance of these techniques. Compared to the benchmark algorithm QMIX, Dueling network and multi-step learning gain some performance improvements. However, the imperceptible impact of Priority replay might be attributed to its sensitivity to hyperparameters, which have not yet been sufficiently fine-tuned. This study aims to deepen the understanding of the impact of the single-agent techniques on MARL. In summary, this project demonstrates the successful implementation of single-agent techniques on MARL. Moreover, the primary methodology within this project is modular optimization, which provides a modular perspective for further enhancing the performance and robustness of MARL.

1.2 Background

1.2.1 Deep Q-learning

Q-learning [16] is a foundational off-policy reinforcement learning algorithm. The optimal action-value function $Q^*(s, a)$ is learned through iterative updates based on the Bellman optimality equation. The Tabular Q-learning, the original form of Q-learning, stores the Q-function in a lookup table over state-action pairs. The update rule is as follows:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

This approach performs well in small-scale environments with discrete state-action spaces. However, it fails in continuous or high-dimensional spaces, where representing all state-action pairs is impossible.

To solve this scalability issue, DQN [17] is proposed as a form of approximate Q-learning. The deep neural network with parameter θ is used to approximate the Q-function, i.e., $Q(s, a; \theta)$. DQN is trained by minimizing the TD error using transitions sampled from a replay buffer D , with the following loss function:

$$L_i(\theta_i) = \mathbb{E}_{(s, a, r, s') \sim U(D)} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta_i^-) - Q(s, a; \theta_i) \right)^2 \right]$$

The target network parameters θ_i^- are updated with the Q-network parameters θ_i every fixed number of steps, improving the training stability. This formulation makes it possible for DQN to handle large-scale and high-dimensional tasks.

1.2.2 QMIX

IQL instructs agents to learn the Q-function independently using their observations and treat other agents as part of the environment. As other agents update their policies, the environment becomes non-stationary [18].

To address this issue, VDN [11] has proposed that the total action-value function can be factorized into a sum of agent utilities:

$$Q((h^1, h^2, \dots, h^d), (a^1, a^2, \dots, a^d)) \approx \sum_{i=1}^d \tilde{Q}_i(h^i, a^i)$$

This formulation enables centralized training by using the team reward signal and ensures decentralized execution across agents. Nevertheless, the additive decomposition of the joint Q-function restricts the expressiveness. VDN does not take full advantage of centralized training because it ignores the extra state information during the training phase.

Thus, QMIX [12] introduces the following enhancements upon VDN:

Mixing Network: QMIX modifies the factorization of the joint Q-value function by replacing the linear sum of individual Q-values with a non-linear combination. Simultaneously, QMIX enforces the monotonicity of the relationship between Q_{tot} and Q_a :

$$\frac{\partial Q_{tot}}{\partial Q_a} \geq 0, \forall a \in A$$

This constraint guarantees that each agent has a positive contribution to the joint Q-value. As a result, the global argmax applied to Q_{tot} yields the same consequence as performing individual argmax operations on each Q_a :

$$\operatorname{argmax}_u Q_{tot}(\tau, u) = \begin{pmatrix} \operatorname{argmax}_{u^1} Q_1(\tau^1, u^1) \\ \vdots \\ \operatorname{argmax}_{u^n} Q_n(\tau^n, u^n) \end{pmatrix}$$

The use of the mixing network allows QMIX to be applied to complicated cooperative environments by capturing the dependencies among agents. This design ensures the individual action selection optimizes the global strategy.

Hypernetwork: QMIX introduces a hypernetwork to generate the parameters of the mixing network. The hypernetwork leverages the global state to adjust the weights applied to each Q-value. The non-negative weights guarantee the monotonicity constraint, thereby maintaining consistency between the local optimal actions and the global optimal joint action.

1.2.3 Rainbow

The Rainbow paper offers a comprehensive extension to DQN, combining a set of improvements that have been proven to be effective in isolation. Its success illustrates the potential of integrating these techniques into this project. Among those, Dueling network, multi-step learning, and Priority replay are implemented, tested, and evaluated, as the focus of this study.

Dueling Network: The Dueling network [19] architecture incorporates two separate estimators within the Q-network: one is responsible for the state-value function $V(s)$ that computes the value of being in the state s , the other is responsible for the advantage function $A(s, a)$ that computes the advantage value of each action. These two outcomes are then combined to form the final Q-value:

$$Q^*(s, a) = V^*(s) + A^*(s, a) - \max_a A^*(s, a)$$

The Dueling network architecture increases learning efficiency, particularly in scenarios where the choice of action has little influence on the outcome. By isolating the estimation of state-

value and advantage, the architecture prevents the mutual interference, resulting in more accurate value estimations.

Multi-step Learning: In conventional TD learning methods, the value updates rely on one-step learning, where the target is calculated according to the immediate reward. However, the technique may lead to low reward signal propagation and convergence efficiency. Thus, Multi-step learning [20], also referred to as n-step learning, is proposed to mitigate these issues by integrating multiple future rewards:

$$y_t = \sum_{i=0}^{m-1} \gamma^i \cdot r_{t+i} + \gamma^m \cdot \max_a Q^*(S_{t+m}, a)$$

This method extends the traditional TD methods, enabling faster reward propagation and better convergence, especially in environments with delayed rewards.

Priority Replay: In DQN, the experience replay buffer is a vital component that improves data efficiency and training stability. In general, the sample strategy follows the uniform distribution within the replay buffer, which indicates all transitions have the identical importance. Nevertheless, this sample strategy neglects the fact that some transitions might have greater importance than others.

Therefore, Priority Experience Replay (PER) [21] solves this limitation by sampling transitions with different priorities, where transitions with higher TD error will have higher probabilities to be sampled. The sampling priority of a transition i is assigned using its TD error plus a bias:

$$p_i = |\delta_i| + \varepsilon$$

Here, δ_i is the TD error, and $\varepsilon > 0$ is a small bias to guarantee all transitions have non-zero probability.

The probability formulation of sampling a transition is as follows:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$$

The parameter $\alpha \in [0,1]$ determines the degree to which TD error affects the probability. Due to the sampling process introducing bias, PER has to apply importance sampling (IS) weights during optimization to compensate for the bias introduced by non-uniform distribution:

$$\omega_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta$$

The IS weight is controlled by β , which anneals gradually to 1. In the early stage of training, a smaller β is used to prioritize samples with higher TD error, making the model focus on more informative transitions. During the training process, the reliance on such emphasis reduces, and a larger β is applied to diminish the bias and force the training approach to the true, uniform distribution.

1.2.4 PyMARL

To facilitate reproducible research in cooperative MARL, the PyMARL framework [22] was released together with the SMAC benchmark. PyMARL is a modular and extensible framework, developed in PyTorch, providing high-quality implementations of representative algorithms, such as QMIX. In addition, the framework organizes the agent networks, mixing networks, replay buffers, and training loops into modular components, making it convenient for flexible customization and construction of new algorithms. The framework also provides evaluation

logging, facilitating researchers to better review experiment settings, monitor data, and analyze results.

1.2.5 Predator-Prey

The Predator-Prey [23] task is a partially observable, cooperative MARL benchmark, designed to evaluate the coordination capabilities of agents under sparse reward signals and inter-agent dependencies environments. A crucial rule is that a successful capture requires two agents to take action on the same prey from adjacent positions simultaneously. This indicates a challenge where agents must learn to synchronize their actions effectively, only utilizing limited local information. It is a rigorous testbed for checking whether the MARL algorithms can desirably model the inter-agent dependencies under partial observation. The core parameter details will be explained in the design section.

1.2.6 Barkla

All experiments were executed on the Barkla high-performance computing (HPC) cluster provided by the University of Liverpool. Every experiment was submitted as Slurm job scripts, specifying GPU requirement, time limitation, runtime environment, and execution script etc. Utilizing the computing cluster enables large-scale computation and facilitates the execution of a large number of experiments.

2 Design

2.1 Design Overview

The overall goal of this project is to enhance the performance of the QMIX by incorporating single-agent techniques. Thus, this project retains the original mixing network architecture while focusing on modifications to other components of the model. Specifically, the architecture of the agent network, TD target, and experience replay buffer are modified. The modification follows the modular structure within the PyMARL framework, which means the Agent (responsible for local Q-value functions), Learner (Managing the training process and computing TD-related values), Run (Controlling the training loops), and Replay Buffer (Storing transitions and regulating the sampling mechanism) components are optimized in this project.

The design section will mainly discuss how these algorithms are incorporated into QMIX, as well as the parameter settings of these algorithms. Furthermore, the section also explains the meanings and configurations of the environment variables used in the experiments, along with the underlying rationale for choosing these values.

2.2 Dueling Network

Integrating the Dueling network into DQN has been shown to improve the accuracy, stability, and robustness of Q-value estimation. Accordingly, incorporating this technique into QMIX is motivated by a similar goal. This technique decomposes the Q-value into two distinct streams:

the state-value estimator and advantage-value estimator. The state-value estimator within the decomposition aims to assess the desirability of the different states and mitigate the impact of noise or transiently high Q-values from suboptimal actions. Moreover, in many states, the choice of action has little influence on the overall return, and directly estimating Q-values in these scenarios might cause low learning efficiency.

In the Predator-Prey task, the importance of agent action selection is emphasized when the agent is close to the prey. In contrast, when the prey is not within the observation of agent, the choice of action becomes largely irrelevant. In these situations, an accurate estimation of the baseline state-value plays a vital role because it can make the advantage function concentrate more on subtle gaps among actions.

There is an important theoretical justification related to the dueling network formulation that is worth mentioning. According to the Bellman optimality principle, the optimal state-value function is stated as the maximum of the optimal action-value function:

$$V^*(s) = \max_a Q^*(s, a)$$

The definition of the optimal advantage function:

$$A^*(s, a) = Q^*(s, a) - V^*(s)$$

A new formulation can be deduced by taking the maximum over both the optimal advantage function and the action-value function:

$$\max_a A^*(s, a) = \max_a Q^*(s, a) - V^*(s)$$

Substitute the expression $V^*(s)$ into $\max_a Q^*(s, a)$ in the above equation:

$$\max_a A^*(s, a) = 0$$

As a result, the final expression can be obtained as follows:

$$Q^*(s, a) = V^*(s) + A^*(s, a) - \max_a A^*(s, a)$$

Therefore, subtracting the max does not affect the correctness of the original equation, and this normalization term serves an important function during training. Without this term, the value and advantage function might fluctuate during training due to the non-identifiable problem. Hence, this term is an indispensable part of this algorithm.

The original paper [19] also mentioned that $\max$ could be replaced with *mean* to improve the training performance. However, owing to the insufficient theoretical justification, this project insists on the original formulation.

To generate the state-value function and the advantage function, the original fully connected layer behind the Recurrent Neural Network (RNN) was replaced with two parallel fully connected layers. These layers offer the parameters of these two functions. In other words, the two branches share a sample backbone network. This design enhances the training performance without significantly increasing the computational workload. Notably, the network architecture modification is the only change introduced by this algorithm, indicating the rest of the training process remains intact, which means the approach is more convenient to implement and reproduce. Overall, this algorithm improves performance while maintaining architectural simplicity and implementation convenience.

2.3 Multi-step Learning

Multi-step Learning has been proven can accelerate the convergence in the learning process. Hence, this project attempts to integrate this technique into QMIX to speed up convergence. One-step learning, used in traditional DQN, can only propagate rewards back one step at a time, slowing down the learning process. Since agents are only awarded when the prey is captured, the Predator-Prey environment only provides sparse rewards. Owing to one-step learning emphasizing short-term reward, it tends to struggle in a sparse-reward environment such as Predator-Prey. Therefore, Multi-step Learning, also known as n-step learning, is implemented in this project to propagate the accumulation of reward signals from multiple future steps. This improvement allows the model to more effectively and rapidly figure out the state-action combinations with higher returns. Another key difference is that one-step learning relies more on bootstrapping during training. Unlike one-step learning, n-step learning mitigates the dependency by integrating actual rewards in the following steps, except the final bootstrapping step. This modification also contributes to enabling efficient training and rapid convergence. Additionally, leveraging rewards from future steps compensates for the partial observation of agents by making agents able to infer meaningful actions according to the long-term return structure.

The following equation defines the TD target in n-step learning:

$$y_t = \sum_{i=0}^{m-1} \gamma^i \cdot r_{t+i} + \gamma^m \cdot \max_a Q^*(S_{t+m}, a)$$

The hyperparameter m indicates the number of future steps used in multi-step learning. This hyperparameter was tested using two settings, 3 and 5, in this project. The main reasons for choosing these two parameter values are as follows:

1. If m is set too small, the algorithm might lose the meaning of multi-step learning and would drift away from the original intention.
2. Choosing a large m may result in high variance due to the accumulation of noise over steps. Additionally, large m may lead agents to adopt excessively cautious behaviors if the environment has a heavy punishment system, such as the agent receiving a penalty at every step.
3. As demonstrated in [24,25], an appropriate step size m can contribute to better performance compared to the two extremes.

Another hyperparameter, γ , is a discount factor. The further a reward is from the current step, the more it is discounted, and the less it has an impact on the current estimate. This parameter is set as 0.99. The main arguments for choosing this parameter value are as follows:

1. The boundary of discount factor γ is $[0,1]$. When γ is approaching 0, the model concentrates more on short-term rewards. Conversely, when γ is reaching 1, the model focuses more on long-term rewards.
2. In tasks with long-term dependencies, a higher discount factor may more likely improve the performance. Given that the Predator-Prey environment provides delayed rewards, adopting a higher discount factor seems more suitable here.

There exists an interdependent relationship between m and γ , and the best configuration may change depending on the characteristics of the task. This project considers $m = 3$ and $\gamma = 0.99$ as an effective combination. When applying this algorithm to other environments,

it is recommended to start fine-tuning from this configuration and adjust the parameters according to the principles discussed above.

Moreover, since it only involves changes to the TD-related calculation, the method is easy to implement and replicate. Overall, Multi-step learning strengthens agents to cope with sparse and delayed rewards, and benefits the stable training and efficient convergence.

2.4 Priority Replay

In the original Experience Replay Buffer, all transitions have equal probabilities of being sampled. This uniform sampling strategy treats critical and other transitions equally, leading to high frequency in sampling transitions with limited learning value. Consequently, the strategy slows down the training and convergence. To mitigate this problem, Priority Experience Replay has been proposed as an alternative to the traditional Replay Buffer. This algorithm measures the importance of each transition based on its TD error. The transitions with higher TD error will be assigned a higher priority and are more likely to be sampled. This prioritized sampling strategy remarkably improves the performance of the model. Predator-Prey is a sparse-reward environment, probably with constant negative rewards. Agents rarely receive positive rewards, resulting in relatively small TD errors for most transitions. Conversely, transitions involving successful prey captures will be granted large rewards, producing bigger TD errors to help speed up the training process. Thus, these transitions are more valuable to be sampled under the PER scheme. Additionally, PER dynamically allocates learning resources to state spaces that are uncertain or have not yet been effectively learned.

This algorithm involves three hyperparameters. The first hyperparameter is discussed as follows:

$$p_i = |\delta_i| + \varepsilon$$

ε is a small positive bias, guaranteeing every transition remains the probability of being sampled. Two values have been tested in the experiments: 10^{-4} and 10^{-5} . Choosing these values are mainly because:

1. An overlarge ε makes all transitions have pretty high priorities, flattening the distribution closer to uniform, which eventually reduces the effect of the priority replay buffer.
2. If ε is set too small, transitions with low TD errors may never be sampled, depriving their opportunity to be learned.
3. In sparse reward environments, a slightly small ε can increase the density of positive samples.

Another crucial hypermeter is α :

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$$

The hyperparameter α primarily controls the degree to which TD error affects the transition sampling probability. In the experiments, two settings were tested: 0.3 and 0.6. The main rationales are as follows:

1. When $\alpha = 0$, PER will degenerate into standard uniform experience replay, eliminating the effect of prioritization. Consequently, excessively low α may invalidate the algorithm and slow down the learning process by failing to focus on informative transitions.
2. For $\alpha = 1$, PER will aggressively concentrate on transitions with higher TD errors. Thus,

setting a larger α corresponds to a more aggressive strategy, which may accelerate learning but also introduces potential risks.

3. The values 0.3 and 0.6 were selected to represent comparatively low and high configurations of α . By comparing these results, the project can obtain insight into which setting is more suitable for the Predator-Prey tasks.

One final but essential formulation of this algorithm is used to calculate the IS weight, which contains an explicit variable and a hidden variable:

$$\omega_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)}\right)^\beta$$

The discussion begins with the explicit variable β . This hyperparameter mainly manipulates the strength of the IS weight correction. A smaller β weakens the correction strength, trading off some bias for faster training. In contrast, a larger β strengthens the correction of the sampling bias at the cost of slower convergence. The basic strategy initializes β with a relatively small value and anneals it gradually to 1. This strategy enables more rapid early training and better convergence in the later phase. In this study, β was initialized at 0.4.

The other latent variable indicates the annealing time of β . It determines the speed at which β increases during training. This parameter was set to the maximum environment time, ensuring β anneals linearly to 1 throughout the training process.

The primary differences between the Priority Replay Buffer and the conventional Replay Buffer are the insert and sample operations.

In the insert function, the main modification is that each newly inserted transition is assigned the maximum priority in the buffer, or a default value of 1 if all existing priorities are zero. This mechanism guarantees that every transition has a high opportunity to be sampled at least once.

In the sample function, the primary change is that the current IS weight, based on the current timestep, is calculated and stored for further computation of loss. In addition, the indices of sampled transitions are also tracked for subsequent priority updates. Naturally, the Priority Replay Buffer adds an extra function to update the priorities of sampled transitions utilizing the recorded indices.

In summary, despite this algorithm introducing the biggest number of modifications in this project, its implementation remains intuitive and replicable. Theoretically, the Priority Replay Buffer enables both fast early-stage learning and stable later-stage convergence.

2.5 Predator-Prey

This section will primarily demonstrate the detailed rules and settings of the Predator-Prey tasks, along with the definitions and functions of the key parameters that were modified in this study.

The Predator-Prey tasks take place on an $n \times m$ grid, where N agents try to capture M stags and M' hares. There is a restriction that every entity cannot occupy the same position. This project uses a fixed size of 10×10 grid. To train cooperative actions among agents, the mechanism restricts that the stags can only be successfully captured if two or more agents execute the 'catch' action. In contrast, hares can be captured by a single agent. Each agent has

an action space $\mathcal{A}_a$ of size 10:

$$\mathcal{A}_a = \{right, down, left, up, stay, catch, look_{right}, look_{down}, look_{left}, look_{up}\}$$

Each prey has an action space $\mathcal{A}_p$ of size 5:

$$\mathcal{A}_p = \{right, down, left, up, stay\}$$

Movements are executed according to the order that predators act first and then prey act. Within the predator and prey groups, the order of execution is randomized. There is a special case where a prey is surrounded by adjacent agents. In this situation, the prey is deemed to be captured due to the lack of available actions. For agents that participate in capture, they will be frozen until the episode limit is reached or all prey are captured. The episode limit refers to the maximum number of time steps per episode, and the mis-capture punishment means the penalty received by an agent for performing a capture action without successfully capturing any prey. The configuration of 'capture_freezes' and 'miscapture_punishment' aims to promote agents to treat capture actions more cautiously. Furthermore, a parameter, 'reward_time', indicates the fixed reward that is given each time step. The purpose of this parameter is to compel agents to accomplish the capture task in fewer time steps. The parameters mentioned above are significant in forming the environments and are expected to have a remarkable influence on the experimental results.

Some other parameters have been modified in this study as well. The observation region, 'agent_obs', is the $[2n + 1, 2n + 1]$ grid with itself as the center. Each agent has a two-channel observation matrix. One channel represents the observation of other agents, and the other shows the observation of prey. This matrix is then squeezed into a one-dimensional feature vector for subsequent use. R_s and R_h , 'reward_stag' and 'reward_hare', mean the reward obtained upon successfully capturing the corresponding prey. Moreover, P_s and P_h indicate the probability that a stag or a hare chooses to stay in the current position. 't_max' denotes the total number of environment time steps over the entire experiment.

The complete parameter lists are included in the appendix.

2.6 Experimental Design

In this project, two types of comparative experiments are conducted. The first kind of experiments focuses on comparing the original QMIX with optimized QMIX under the same environments. In this type of experiment, various parameter settings of the optimization algorithms are tested to determine the optimal parameter configuration in the current environment. The second kind of experiments concentrates on researching the impact of environmental parameters on the performance of optimization algorithms. In this type of experiment, the algorithm configuration will remain unchanged. Conversely, the environment parameters are modified and experimented with to summarize the regular patterns in the adaptability of the optimization algorithms to different environments. Constrained by the most critical environmental parameters, 'return_mean' and 'ep_length_mean' are considered the two most important metrics for evaluating the performance of the model. 'return_mean' is an intuitive evaluation metric because it directly represents the cumulative reward the entire system received. Unlike 'return_mean', 'ep_length_mean' indirectly assesses the effectiveness of the system by evaluating the episode time the entire system used.

2.6.1 Comparative Experiments

To comprehensively evaluate the performance improvements and environmental adaptability brought by single-agent techniques to the QMIX algorithm, this project designs two sets of comparative experiments. These experiments not only examine the effectiveness of each technique under different parameter configurations, but also explore the underlying relationship between the optimized algorithms and environmental characteristics.

The first category of experiments focuses on algorithm comparison within a fixed environmental setting. This part of the experiments can be separated into two parts: determining the optimal parameter configurations of optimized algorithms, and then comparing the performance of optimized algorithms under these optimal parameter configurations with the standard QMIX. Since the Dueling Network does not contain tunable hyperparameters, the optimized algorithm is compared with the original QMIX algorithm directly. For Multi-step Learning, this project mainly investigates how the number of steps influences the performance. More specifically, the 3-step and 5-step settings were tested in this experiment. In the Priority Replay Buffer experiment, three different parameter combinations $(\alpha, \beta, \varepsilon)$ were examined. These three parameter combinations are as follows: $(0.3, 0.4, 10^{-4})$, $(0.6, 0.4, 10^{-4})$, $(0.3, 0.4, 10^{-5})$. The baseline setting $(0.6, 0.4, 10^{-4})$ was used to determine the other parameter combinations by modifying one parameter at a time.

The second category of experiments concentrates on investigating the latent relationships between the optimized algorithms and environmental features. In light of its better performance in previous experiments and more complex underlying mechanism, the multi-step learning is considered more valuable for further exploration in this part of the study. Therefore, it was selected as the invariant algorithm in this category of experiments. The influence of environmental factors on the improved algorithm is revealed by comparing it with the QMIX baseline. The naive comparison of results across different environments is meaningless and can result in incorrect interpretations. The 'predator_preym.yaml' configuration is the baseline task in the category of experiments. Two comparative experiments that conducted in this project, modified the number of prey and the magnitude of the time penalty. By analyzing the performance trends across different environments, this project aims to summarize the generalization, compatibility, and transferability of the model.

These two categories of experiments complement each other, highlighting the insights from different angles into the improvement techniques. Combining these two experimental designs ensures the conclusions are on both performance and environment, making it more comprehensive for understanding multi-agent reinforcement learning algorithm optimization.

2.6.2 Evaluation Metrics

To objectively evaluate the effectiveness of the optimized algorithms, two primary evaluation metrics, 'return_mean' and 'ep_length_mean', are used in this study. These two metrics have the strongest association with the global performance of a multi-agent model. Additionally, these metrics are tightly relevant to some of the more important parameters in the Predator-

Prey task settings, reflecting how environmental configuration affects the performance.

The definition of the 'return_mean' metric is the average cumulative reward the system obtains across multiple episodes. This metric directly reflects the performance of the model, as well as the capacity of the multi-agent system under its current policy. A higher value of 'return_mean' indicates a greater ability to gain rewards, meaning stronger model performance. Conversely, a lower value reflects the opposite. Furthermore, key environmental parameters, 'capture_freezes', 'miscapture_punishment', and two prey rewards, are highly associated with this metric, further emphasizing the importance of this metric.

Another metric, 'ep_length_mean', an indirect measurement, represents the average episode length across multiple episodes. A shorter episode length usually argues that agents can accomplish the tasks faster and more efficiently, and vice versa. The key environmental parameter, 'reward_time', is tightly linked with this metric, also demonstrating the significance of this metric.

Metric 'return_mean' is used to assess the quality of the policy, while Metric 'ep_length_mean' evaluates the efficiency of the policy. These two metrics are complementary, jointly establishing a comprehensive evaluation framework. These metrics are recorded at regular intervals during the training process. Every experiment was run five times with different random seeds to diminish the influence of stochasticity in the results. These results were aggregated and plotted with a line chart to facilitate further analysis of the training performance and convergence condition.

3 Implementation

3.1 Overview of the Implementation

This project primarily implements the integration of Dueling Network, Multi-step Learning, and Priority Replay Buffer into the QMIX to boost its performance. The entire implementation is built on the open-source PyMARL framework, developed by DeepMind for cooperative MARL. The modular architecture of PyMARL significantly convenient the integration of new parameter configuration files, agent networks, TD update mechanisms, and enhanced replay buffers, playing a critical role in the implementation of this project. In addition, the framework utilizes the 'sacred' library in Python to log experiments, including parameter configurations and runtime results, facilitating the subsequent review and analysis.

The following table outlines the general description and the corresponding location of each modification in this project:

Modification	Description	File Location
Dueling Network	Agent Network Modification	modules/agents/dueling_agent.py modules/agents/__init__.py
Multi-step Learning	TD Calculations Update	learners/q_learner.py
Priority Replay Buffer	Sampling Strategy Optimization	components/episode_buffer.py learners/q_learner.py run.py
Algorithmic Configuration	Algorithm Parameter Configuration	config/algs/*.yaml
Environmental Configuration	Predator-Prey Environment Configuration	config/envs/*.yaml
Experiment Configuration Log	All parameter configurations Log	results/sacred/**/config.json
Experiment Result Log	All results Log	results/sacred/**/info.json
Visualization	Plotting Script	results/sacred/draw.py

Table 1. Overview of Modification and File Locations

3.2 Implementation of Techniques

3.2.1 Dueling Network

In the modification of the agent network, a new class was created under `modules/agents/` to define the new network architecture. Here, the output of the RNN is split into two streams: one for producing the state-value function and the other for generating the advantage function.

Code 1: Dueling Network

Network Architecture

```
self.fc1 = nn.Linear(input_shape, args.rnn_hidden_dim)
self.rnn = nn.GRUCell(args.rnn_hidden_dim, args.rnn_hidden_dim)

self.fc_value = nn.Linear(args.rnn_hidden_dim, 1)
self.fc_advantage = nn.Linear(args.rnn_hidden_dim, args.n_actions)
```

Forward Propagation

```
value = self.fc_value(h)
advantage = self.fc_advantage(h)
q = value + advantage - advantage.max(dim=-1, keepdim=True).values
```

Observing the final outputs of the network, it is obvious that the state-value function produces a single scalar representing the value of the current state. In addition, the advantage function outputs a one-dimensional vector of length equal to the number of agent actions, illustrating the corresponding value of each action in the given state. In the forward propagation process, the maximum advantage value is subtracted from Q_{tot} to guarantee the identifiability.

After implementing the algorithmic modification, the new agent network has to be registered in `modules/agents/__init__.py` in order to be used within the framework.

3.2.2 Multi-step Learning

The `q_learner.py` file contains all the modifications of the multi-step learning technique. The code below shows the implementation of multi-step learning.

Code 2: Multi-step Learning

```
n_rewards = th.zeros_like(rewards)
gamma_tensor = th.tensor([self.args.gamma**i for i in range(N)],
dtype=th.float, device=n_rewards.device)
steps = mask.flip(1).cumsum(dim=1).flip(1).clamp_max(N).long()
for i in range(batch.max_seq_length - 1):
    n_rewards[:, i, 0] = ((rewards * mask)[:, i:i+N, 0] *
        gamma_tensor[:, (batch.max_seq_length - 1 - i)]).sum(dim=1)
indices = th.linspace(0, batch.max_seq_length-2,
steps=batch.max_seq_length-1,
device=steps.device).unsqueeze(1).long()
n_targets_terminated = th.gather(target_max_qvals*(1-terminated),
dim=1, index=steps.long()+indices-1)
targets = n_rewards + th.pow(self.args.gamma, steps.float()) *
n_targets_terminated
```

'n_rewards' is a tensor with the same dimensions as rewards, storing the n-step discounted rewards corresponding to each time step. 'gamma_tensor' is used to precompute the powers of the discount factor γ , allowing the algorithm to retrieve them directly in further computation to improve the computational efficiency. 'step' represents the maximum number of valid future steps for each time step, which is derived from backward searching. Then, use a for loop to calculate the n-step cumulative reward for each time step. Afterwards, an index tensor 'indices' is generated to assist in selecting the correct Q-value while computing the 'n_target_terminated'. Subsequently, calculates the bootstrapped Q-values for each step and masks out terminal states to avoid bootstrapping beyond the episode ends. Eventually, the TD target is obtained by adding the bootstrapped Q-value and 'n_rewards'.

The computation process is not the hardest part. The primary difficulties during the implementation are as follows:

1. The tensors involved differences in the shapes and sizes. Special attention needs to be paid to correctly utilize the relevant parts.
2. All tensors must be placed on the same device for operations to execute correctly. If they are on different devices, they must be transferred to the same device in advance to avoid runtime error.

3.2.3 Priority Replay Buffer

The main changes for the Priority Replay Buffer are implemented in the `episode_buffer.py` and `q_learner.py` files. The modifications in `run.py` are meant to initialize a new Priority Replay Buffer to replace the original Replay Buffer. The following are the key code segments related to the modification:

Code 3: Priority Replay Buffer

Insert

```
max_priority = self.priorities.max() if self.episodes_in_buffer >
0 else 1.0
for index in insert_indices:
    self.priorities[index] = max_priority
```

Sample

```
scaled_priorities = self.priorities[:self.episodes_in_buffer] **
self.prb_alpha
probability = scaled_priorities / scaled_priorities.sum()
indices = np.random.choice(self.episodes_in_buffer, batch_size,
p=probability)
beta = self.prb_beta + (1.0 - self.prb_beta) * beta_time /
self.prb_beta_annual_time
total = self.episodes_in_buffer
weights = (1 / (total * probability[indices])) ** beta
weights /= weights.max()
batch = self[indices]
batch.is_weights = th.tensor(weights, dtype=th.float64,
device=self.device)
batch.indices = indices
```

Update

```
for i, error in zip(indices, td_errors):
    self.priorities[i] = np.abs(error).mean() + self.prb_eps
```

Loss Calculation

```
is_weights =
batch.is_weights.unsqueeze(1).unsqueeze(2).expand_as(td_error)
loss = ((masked_td_error ** 2) * is_weights).sum() / mask.sum()
```

To implement this improvement, a class called `PriorityReplayBuffer` needs to be created. There are two possible methods: either write a new class or inherit from `ReplayBuffer`. Since inheriting from the original class requires nearly all functions to be overridden. Therefore, to maintain the principle, high cohesion and low coupling, a completely new class was implemented in this project. Additionally, there is an important point that the priorities update must be applied after the back propagation. A similar issue encountered during the implementation, as mentioned in multi-step learning, is that the buffer is on the CPU while the TD error is placed on the GPU, which contributes to a runtime error.

3.3 Runtime Environment Configuration and Experiment Execution

This project is executed within an Anaconda virtual environment, which provides an isolated and replicable setup for installing dependencies and managing packages. Owing to the PyMARL framework is relatively old and some of its libraries have been updated, this project does not rely on the original environment configuration provided by the framework. Instead, the dependencies were installed mutually, starting from the PyTorch compatible with the

CUDA version. The other key dependencies were installed using the following command:

Command 1

```
pip install sacred numpy scipy matplotlib seaborn pyyaml pygame  
pytest probscale imageio snakeviz tensorboard-logger
```

However, the framework is still incompatible with two of the newest packages, leading to runtime errors. Therefore, specific versions need to be installed:

Command 2

```
pip uninstall pyyaml  
pip install pyyaml==3.13  
pip uninstall protobuf  
pip install protobuf==3.19.1
```

Despite the environment setup, three issues were encountered during runtime:

1. There is a code error in the original framework:

```
self.logger.log_stat("grad_norm", grad_norm.item(), t_env)
```

Originally, the statement was missing `'.item()'` behind the `'grad_norm'`, which resulted in a type mismatch.
2. As SMAC is not utilized in this project, statements related to it, which would result in errors due to the environment being missing, were commented out.
3. The Predator-Prey environment used in this project is imported from the WQMIX repository. The parameter configuration was initially followed from the README.md file of that project. However, this instruction turned out to be unclear, leading to an incorrect setting. Specifically, the value of `'epsilon_anneal_time'` was set to 1,000,000, resulting in different performance from the results reported in the original paper. After a careful investigation of other parameter configurations, this value was corrected to 50,000.

All experiments were conducted on the university HPC cluster, named Barkla. This cluster runs on the Linux operating system and uses the Slurm job scheduling system. Thus, a Slurm job submission script must be created to run experiments on Barkla. Here is the basic form of the Slurm script:

Slurm Script

```
#!/bin/bash  
#SBATCH --job-name=(name)  
#SBATCH --output=result_%x_%j.output  
#SBATCH --error=error_%x_%j.output  
#SBATCH --time=3-00:00:00  
#SBATCH --partition=gpu  
#SBATCH --gres=gpu:1  
#SBATCH --export=ALL  
#SBATCH --mail-user=(email address)  
#SBATCH --mail-type=ALL  
  
eval "$(conda shell.bash hook)"  
module load libs/cuda/11.3  
conda activate pymarl  
(The path of the '.sh' file) 0
```

The Slurm script specifies the node type, the number of nodes, the maximum runtime, the runtime environment, and the execution script.

Moreover, the project defined execution scripts for different experiments, facilitating the Slurm script to call them without unnecessary modifications. This is an example of the execution script:

Execution Script

```
python /QMIX/src/main.py  
--config=(algorithmic configuration) --env-config=(environmental configuration)
```

3.4 Logging and Evaluation Tool

This project directly utilizes the logging system that is already integrated within the framework. Logs are generated under the `result/sacred/` directory automatically. For convenience of visualization, the logs are categorized according to the used task and the algorithm.

This project developed a drawing tool, `draw.py`, to plot line charts based on recorded data. The program reads the 'return_mean' and 'ep_length_mean' values from five experimental log files, and calculates the maximum, minimum, and mean for each metric. Then, it aligns these statistics with the corresponding environment steps to produce the coordinate points. Finally, these coordinates are plotted and connected to generate the final line. To illustrate the stability level of the algorithm, the region between the maximum and minimum is filled using a lighter color of the corresponding line.

4 Testing & Evaluation

This section primarily analyzes the training performance and convergence behavior of the optimized algorithms based on the plotted curve of two key metrics: 'return_mean' and 'ep_length_mean'. This project is also attempting to explain the latent rationale behind the observations. This section could be divided into two parts: algorithm comparison experiments and environment comparison experiments.

4.1 Algorithm Comparison Experiments

This part mainly assesses the performance of the improvements. For the non-parametric method, Dueling Network, this project directly compares it with QMIX. For improvements that require hyperparameter tuning, a preliminary set of experiments is conducted beforehand to figure out the relatively optimal parameter configurations, which are then used in subsequent comparisons with the original QMIX.

The evaluation begins with the Dueling Network.

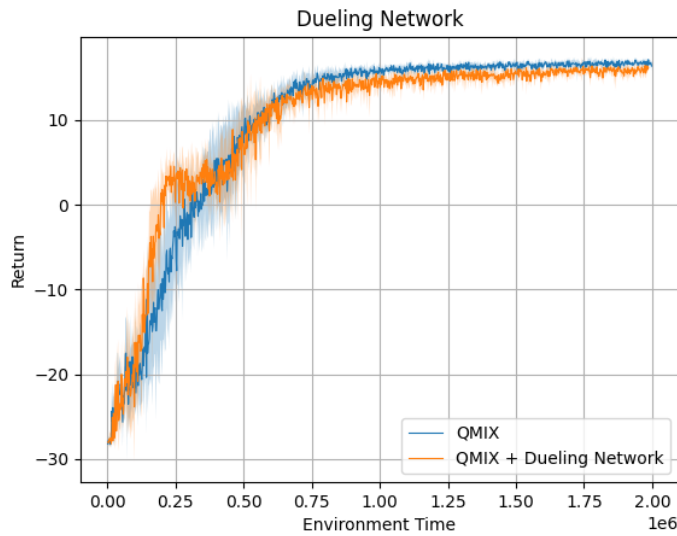


Figure 1: Comparison of Episode Return between QMIX and QMIX with Dueling Network

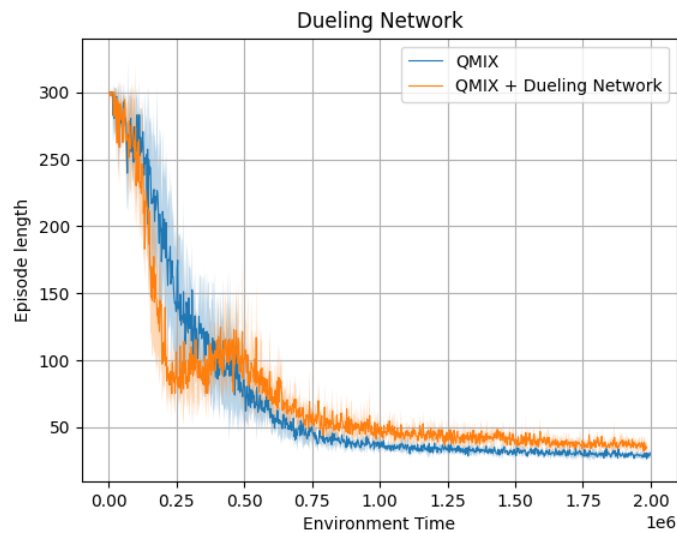


Figure 2: Comparison of Episode Length between QMIX and QMIX with Dueling Network

In Figure 1, the return mean over environment time shows that the dueling network improves QMIX to achieve faster early-stage training speed and comparably stable convergence. In Figure 2, the optimized algorithm consistently has shorter episode lengths earlier in the training process, indicating more efficient policy learning. According to the observation, it can be seen that the Dueling Network QMIX has a narrower shaded area in the middle stage of training, exhibiting its stability during this phase of the training process. During the middle phase of training, the Dueling variant has a temporary performance decrease. This may be owing to the misalignment of the state-value function and the advantage function. Overall, the result argues that QMIX with Dueling Network has more efficient learning and better task completion, especially within the first 0.25 million steps.

The evaluation continues with Multi-step Learning:

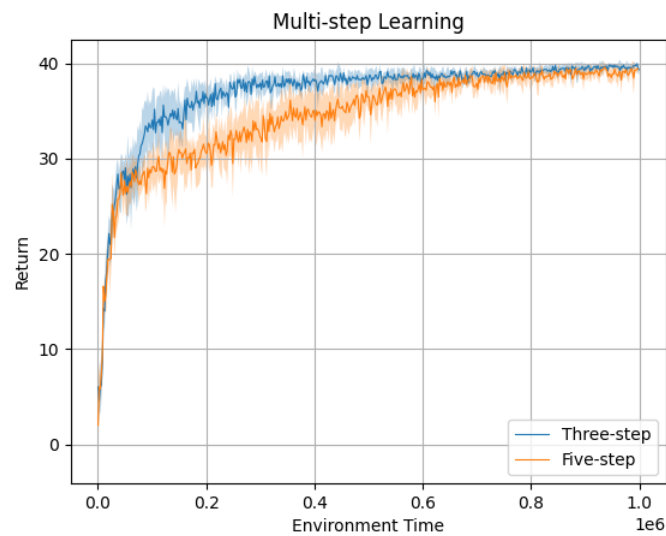


Figure 3: Comparison of Episode Return Between 3-Step and 5-Step Multi-step Learning

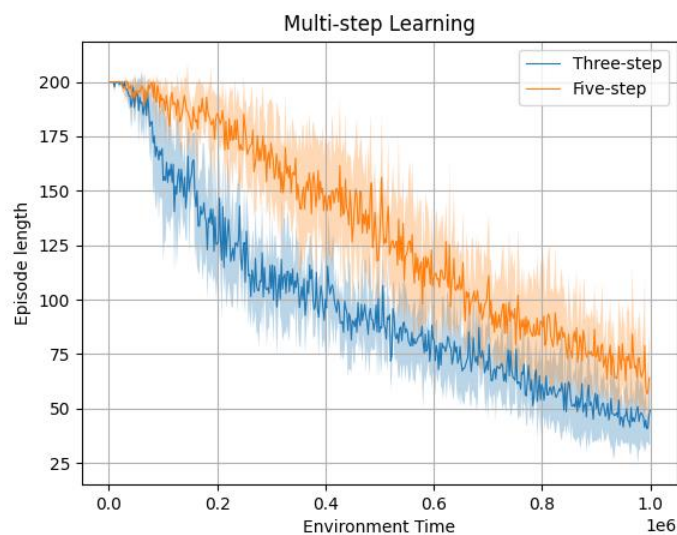


Figure 4: Comparison of Episode Length Between 3-Step and 5-Step Multi-step Learning

This experiment compares the performance of Multi-step Learning using 3-Step and 5-Step in the same environment. As shown in Figure 3, both variants eventually converge to a similar level, but the 3-step version obtains higher training efficiency. Figure 4 further confirms that the 3-step multi-step learning has significantly shorter episode length throughout the entire training process, suggesting better performance. This observation implies that excessively long reward propagation may introduce higher variance and diminish the learning efficiency.

Based on the above experiment, the 3-step Multi-step Learning is selected for comparison with QMIX:

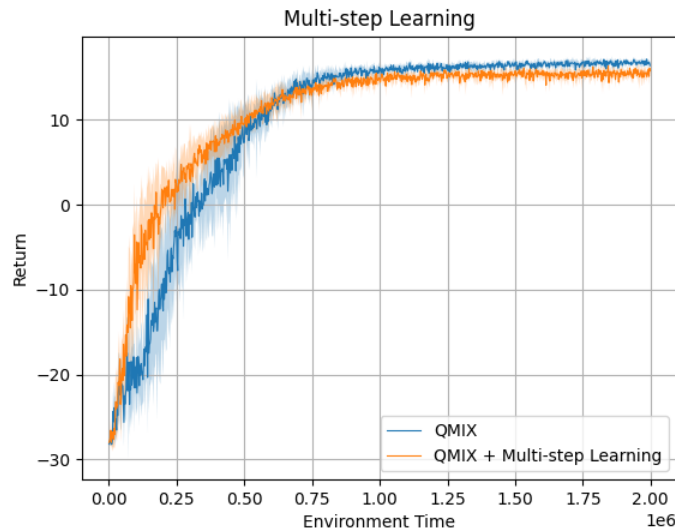


Figure 5: Comparison of Episode Return Between QMIX and QMIX with Multi-step Learning

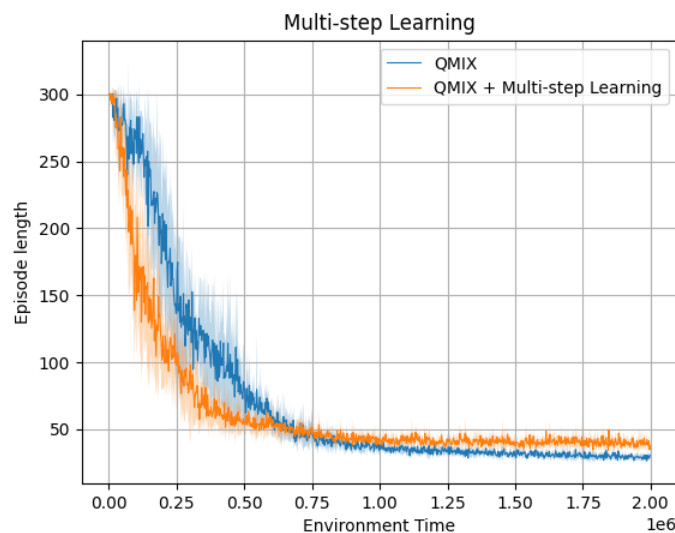


Figure 6: Comparison of Episode Length Between QMIX and QMIX with Multi-step Learning

Figures 5 and 6 present a performance comparison between original QMIX and 3-step Multi-step Learning QMIX. In Figure 5, the return curve of the multi-step learning variant shows faster initial growth and greater stability compared to QMIX. Meanwhile, Figure 6 displays a corresponding decrease in episode length, which could also indicate better training performance.

The symmetry between the return curve and the episode length curve demonstrates that the agents not only learn to capture prey more efficiently but also complete the task in fewer steps. This training result indicates that multi-step can effectively improve QMIX for faster convergence.

The final experiments fall on Priority Replay:

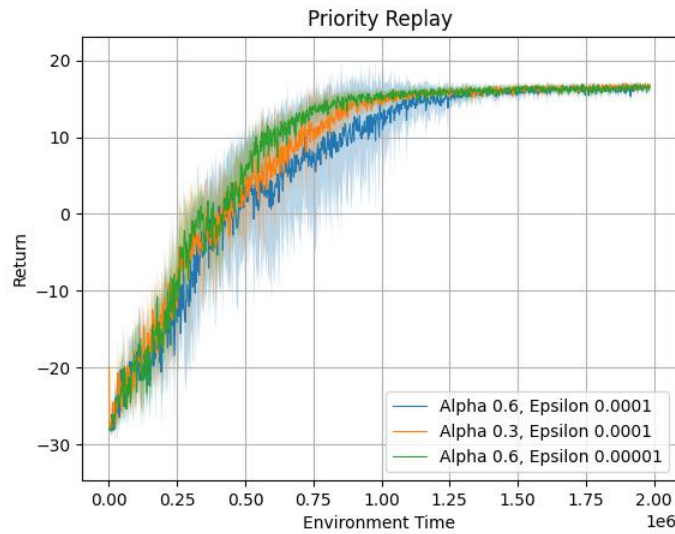


Figure 7: Comparison of Episode Return Among Priority Replay with Varying α and ϵ Settings

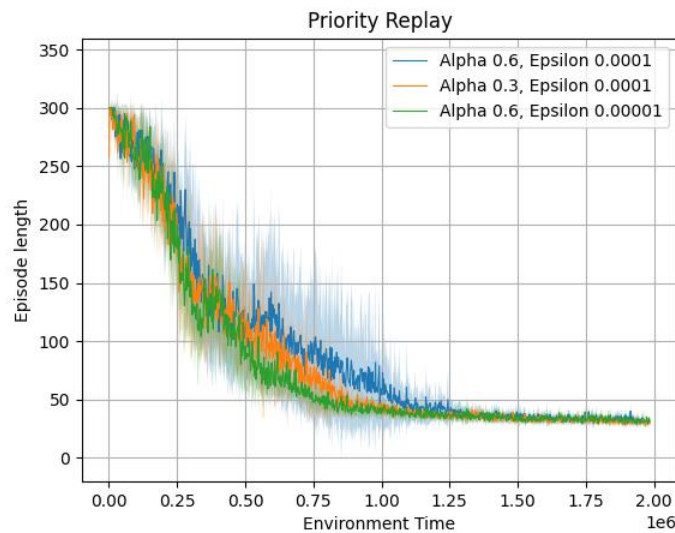


Figure 8: Comparison of Episode Length Among Priority Replay with Varying α and ϵ Settings

Figures 7 and 8 compare the performance of the Priority Replay Buffer under three configurations. Using the configuration, $\alpha = 0.6$ and $\epsilon = 0.0001$, as the baseline, the other two configurations respectively modify α to 0.3 and ϵ to 0.00001.

The return curve in Figure 7 and the length curve in Figure 8 both show that all settings successfully converge to similar final results. Nevertheless, the stability in the middle phase is noticeably different. The configuration with $\epsilon = 0.00001$ has the narrowest shaded region. In contrast, the blue curve with configuration $\alpha = 0.6$ and $\epsilon = 0.0001$ displays the widest shadow. This situation suggests the importance of parameter ϵ on the training stability. On the other hand, the result reflects that this algorithm is highly sensitive to hyperparameter changes.

Based on the above experiment, the priority replay algorithm with configuration, $\alpha = 0.6$ and $\varepsilon = 0.00001$, was selected to compare with QMIX:

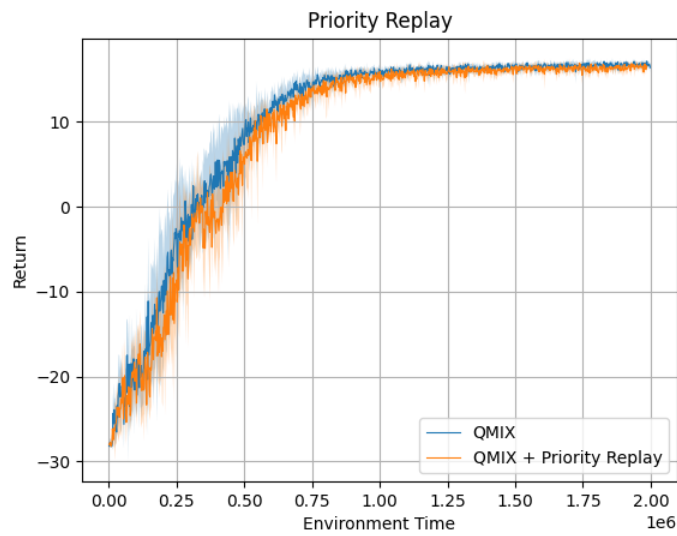


Figure 9: Comparison of Episode Return Between QMIX and QMIX with Priority Replay

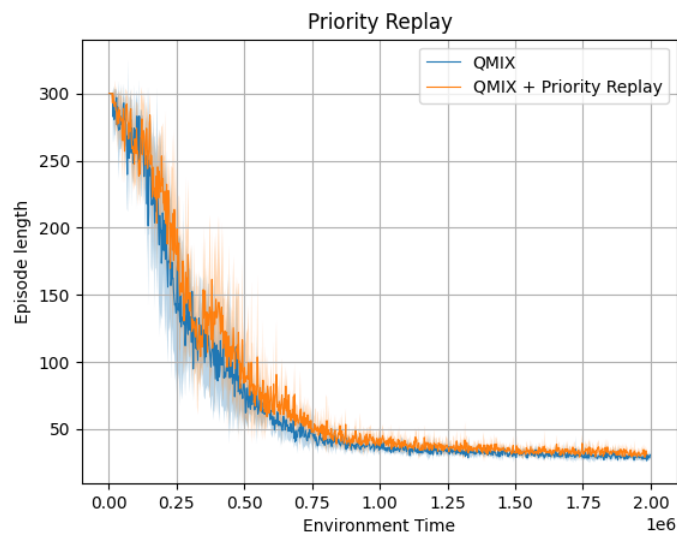


Figure 10: Comparison of Episode Length Between QMIX and QMIX with Priority Replay

In both figures, QMIX and its enhancement, Priority Replay, exhibit extremely similar learning curves, meaning that incorporating the Priority Replay Buffer does not contribute to substantial performance improvement. The only notable difference is the narrower shaded area around the Priority Replay curves, which might indicate a slight stability improvement.

There are two reasonable hypotheses may explain this scenario:

1. Since this algorithm is highly sensitive to hyperparameter change, the optimal hyperparameter configuration has not been discovered.
2. Priority Replay Buffer might introduce additional uncertain factors that might offset its benefits.

4.2 Environment Comparison Experiments

This part mainly assesses the effect of the environment on the Multi-step Learning algorithm.

The first experiment modified the number of stags and hares:

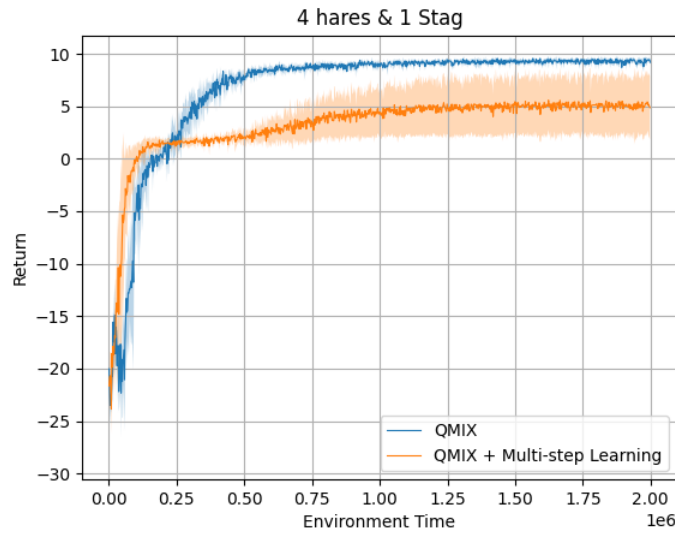


Figure 11: Return Comparison in 4 Hares & 1 Stag Environment

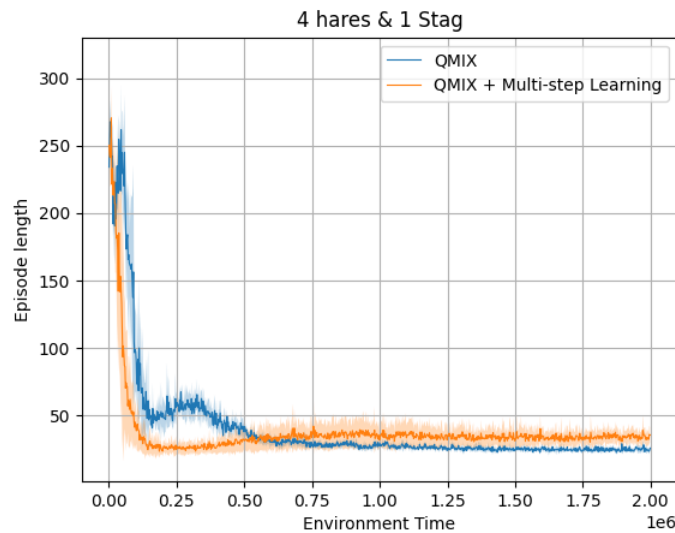


Figure 12: Length Comparison in 4 Hares & 1 Stag Environment

In Figure 11, it can be observed that the multi-step variant converges quickly but is stuck in a suboptimal situation, with a higher likelihood of failure making further progress. Analyzing Figure 12, the multi-step QMIX is capable of rapidly reducing the episode length, but cannot further converge the model in this environment. One possible explanation for this phenomenon is that the continuous time penalties during multi-step reward propagation result in a negative accumulated reward. As a consequence, the model might learn to focus more on immediate rewards rather than long-term benefits.

Another experiment alters the reward time:

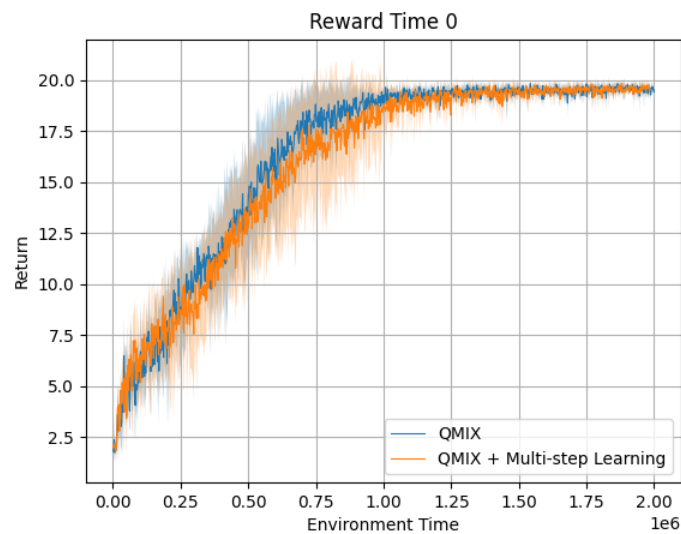


Figure 13: Return Comparison in Reward Time = 0 Environment

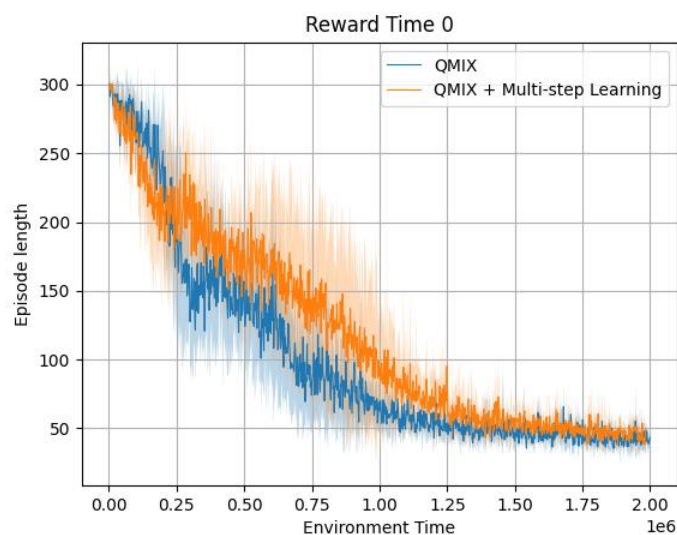


Figure 14: Length Comparison in Reward Time = 0 Environment

Under the condition of reward time = 0, the multi-step variant does not suffer from the suboptimal convergence question seen in the previous setting. However, when compared to the same configuration with a reward time = -0.1, the curve of multi-step QMIX is merely comparable to, or even slightly worse than, the curve of the original QMIX.

5 Project Ethics

This project falls under Data Category A, as it does not involve any data derived from humans or animals. It also belongs to Human Participant Category 0, meaning no human participants were involved at any stage, including requirements analysis or software evaluation. As a theoretical research project conducted independently, no additional ethical considerations are required.

This section does not apply to this project.

6 Conclusion & Future Work

6.1 Conclusion

This project aims to enhance the QMIX algorithm in cooperative MARL by integrating simple single-agent techniques from the Rainbow paper. The implementation incorporates the Dueling Network, Multi-step Learning, and Priority Replay Buffer into the PyMARL framework. All experiments are conducted on the Predator-Prey environments and executed by Barkla, the HPC cluster, to guarantee scalability and replicability.

The results argue that these enhancements contribute to slight performance improvement, faster convergence, and training stability in some configurations of environments. In particular, Dueling Network and Multi-step Learning significantly boost the convergence speed in the early stage of training. Nevertheless, several experiments also indicate that Multi-step Learning may cause the model to get stuck in suboptimal solutions and fail to converge, or perform worse than QMIX in some environments.

The primary contribution of this project is the successful integration of multiple simple single-agent techniques into the QMIX algorithm, leading to measurable performance improvements in several environments. In addition, this project evaluates the algorithm improvements, as well as attempting to explore the underlying relationship between algorithms and environments. Last but not least, this project also provides a modular algorithm improvement perspective.

6.2 Future Work

Further improvements and research directions can be explored in the following aspects:

Integrating additional techniques from Rainbow DQN:

Noisy Networks: Improving the epsilon greedy exploration method

Distributional RL: Modeling the entire returns distribution

Scaling to more environments:

SMAC: More complex environments

Other Predator-Prey configurations: For further comprehension of the techniques

Improving Dueling Network:

$\text{mean}_a A^*(s, a)$: Replacing the $\max_a A^*(s, a)$

Improving Priority Replay:

SumTree: Accelerate the sampling process

More parameter configuration exploration: Fully explore the impact of hyperparameters on the Priority Replay algorithm

7 BCS Project Criteria & Self-Reflection

This project will attempt to meet the requirements of BCS (British Computer Society) and deliver an innovative and promising technique in this region.

1. **Practical and Analytical Skills:** This project focuses on the QMIX multi-agent reinforcement learning algorithm, primarily implemented in Python. It requires not only advanced programming skills but also a solid comprehension of reinforcement learning principles. The analysis aspect involves the ability to evaluate the data, analyze the trend, and infer the underlying rationale of the results.
2. **Innovative and Creativity:** This project will transfer optimization techniques from single-agent algorithms to multi-agent algorithms, modifying the parameters, formulas, and even network structures to achieve better performance. To better reach these targets, it is necessary to review extensive literature, understand new techniques, and try to implement them in this project.
3. **Synthesis Information and Evaluation of Solution:** In this project, the testing and experimental part repeats these two sections: Design experiments by adjusting algorithmic or environmental parameters, followed by conducting the experiments and evaluating the results. This cyclical process emphasizes the integration and analysis of information, ideas, and applications.
4. **Fulfilling a Broader Need:** Multi-agent reinforcement learning has a wide range of applications in the real world, including issues such as robotic collaboration. In this project, performance optimization guarantees greater robustness and accuracy for devices. Simultaneously, multi-agent optimization approaches can promote further research in this area and offer inspiration for research in other fields.
5. **Self-Management of Major Work:** One of the major difficulties I encountered in this project was related to project management, including task scheduling and time allocation. Although I set a project plan in advance, the overall execution did not fully adhere to the original schedule. One contributing factor may be that I have limited related project experience, which made it difficult to accurately estimate the time required for each component. I underestimated the difficulty of many aspects of the project, which resulted in a longer time than expected. For example, my literature review was slower than expected, since fully understanding the content often requires multiple readings of each paper and additional sources to assist comprehension, such as blog articles. In addition, a considerable amount of time was spent on environment setup due to the unclear instructions. Another reason is that I found my project was frequently affected by unexpected events or other assignments. My time allocation across different tasks was not ideal. I am often deeply immersed in one project while completely neglecting another one. This might result in additional time costs to reacquaint myself with the postponed project. I also recognize that a significant shortcoming was my tendency to underestimate the potential impact of unexpected events on the project. For instance, I did not anticipate that the maintenance of the university server would have such a significant

impact on my project before I began running the experiments. At the beginning of the experimentation phase, a storage failure on the server triggered maintenance, which led to unstable server connectivity. Subsequently, this led to full maintenance for the Barkla HPC server, where only a limited number of nodes remained accessible, further constraining experimental resources. Nevertheless, I believe I also demonstrated some strengths. In response to disruptions or deviations from the original plan, I was generally able to reorganize tasks efficiently and make adjustments in time to minimize the loss.

6. Critical Self-Evaluation: This project also revealed other weaknesses of mine. I came to realize that my ability to resolve problems was not as strong as expected. When facing problems caused by incorrect parameter settings, I was unable to identify the appropriate solution, which led to a loss of three days. Additionally, I often lacked the initiative to seek help or feedback from others when encountering difficulties, which further delayed my progress. Moreover, I recognize that my experimental capacity requires further improvement. In addition, my reasoning and analytical skills also need improvement. In the final experiment of the project, I was unable to provide a reasonable explanation for the abnormal results. These experiences have made me more aware of the importance of proactive communication and continuous reflection throughout a project. However, I think one of my key advantages is my persistent learning attitude, which motivates me to continually seek out, understand, and internalize new knowledge. With this mindset, I am confident that the weaknesses identified throughout this project can be progressively overcome.

8 References

- [1] P. Long *et al.*, "Towards Optimally Decentralized Multi-Robot Collision Avoidance via Deep Reinforcement Learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2018, pp. 6252–6259, doi: [10.1109/ICRA.2018.8461113](https://doi.org/10.1109/ICRA.2018.8461113).
- [2] Y. Chen *et al.*, "MA4DIV: Multi-Agent Reinforcement Learning for Search Result Diversification," *arXiv preprint arXiv:2403.17421*, 2024. [Online]. Available: <https://doi.org/10.48550/arxiv.2403.17421>.
- [3] Y. Cao *et al.*, "An Overview of Recent Progress in the Study of Distributed Multi-Agent Coordination," *IEEE Trans. Ind. Informatics*, vol. 9, no. 1, pp. 427–438, Feb. 2013, doi: [10.1109/TII.2012.2219061](https://doi.org/10.1109/TII.2012.2219061).
- [4] M. Hüttenrauch, A. Šošić, and G. Neumann, "Guided Deep Reinforcement Learning for Swarm Systems," *arXiv preprint arXiv:1709.06011*, 2017. [Online]. Available: <https://doi.org/10.48550/arxiv.1709.06011>.
- [5] D. Mguni *et al.*, "Coordinating the Crowd: Inducing Desirable Equilibria in Non-Cooperative Systems," in *Proc. 18th Int. Conf. Autonomous Agents MultiAgent Syst. (AAMAS)*, Richland, SC, 2019, pp. 386–394, doi: [10.5555/3306127.3331718](https://doi.org/10.5555/3306127.3331718).
- [6] J. Wang *et al.*, "QPLEX: Duplex Dueling Multi-Agent Q-Learning," *arXiv preprint arXiv:2008.01062*, 2020. [Online]. Available: <https://doi.org/10.48550/arxiv.2008.01062>.

- [7] F. A. Oliehoek, M. T. J. Spaan, and N. Vlassis, "Optimal and Approximate Q-value Functions for Decentralized POMDPs," *J. Artif. Intell. Res.*, vol. 32, pp. 289–353, 2008, doi: [10.1613/jair.2447](https://doi.org/10.1613/jair.2447).
- [8] L. Kraemer and B. Banerjee, "Multi-agent reinforcement learning as a rehearsal for decentralized planning," *Neurocomputing*, vol. 190, pp. 82–94, May 2016, doi: [10.1016/j.neucom.2016.01.031](https://doi.org/10.1016/j.neucom.2016.01.031).
- [9] R. Lowe et al., "Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments," *arXiv preprint arXiv:1706.02275*, 2017. [Online]. Available: <https://doi.org/10.48550/arxiv.1706.02275>.
- [10] J. Foerster et al., "Counterfactual Multi-Agent Policy Gradients," *arXiv preprint arXiv:1705.08926*, 2017. [Online]. Available: <https://doi.org/10.48550/arxiv.1705.08926>.
- [11] P. Sunehag et al., "Value-Decomposition Networks for Cooperative Multi-Agent Learning," *arXiv preprint arXiv:1706.05296*, 2017. [Online]. Available: <https://doi.org/10.48550/arxiv.1706.05296>.
- [12] T. Rashid et al., "QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning," *arXiv preprint arXiv:1803.11485*, 2018. [Online]. Available: <https://doi.org/10.48550/arxiv.1803.11485>.
- [13] A. Mahajan et al., "MAVEN: Multi-Agent Variational Exploration," *arXiv preprint arXiv:1910.07483*, 2019. [Online]. Available: <https://doi.org/10.48550/arxiv.1910.07483>.
- [14] T. Rashid et al., "Weighted QMIX: Expanding Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning," *arXiv preprint arXiv:2006.10800*, 2020. [Online]. Available: <https://doi.org/10.48550/arxiv.2006.10800>.
- [15] M. Hessel et al., "Rainbow: Combining Improvements in Deep Reinforcement Learning," *arXiv preprint arXiv:1710.02298*, 2017. [Online]. Available: <https://doi.org/10.48550/arxiv.1710.02298>.
- [16] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, no. 3–4, pp. 279–292, May 1992, doi: [10.1007/BF00992698](https://doi.org/10.1007/BF00992698).
- [17] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015, doi: [10.1038/nature14236](https://doi.org/10.1038/nature14236).
- [18] C. Claus and C. Boutilier, "The dynamics of reinforcement learning in cooperative multiagent systems," in *Proc. 15th Nat. Conf. Artif. Intell. and 10th Innovative Appl. Artif. Intell. Conf. (AAAI/IAAI)*, Madison, WI, USA, Jul. 1998, pp. 746–752, doi: [10.5555/295240.295800](https://doi.org/10.5555/295240.295800).
- [19] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling network architectures for deep reinforcement learning," *arXiv preprint arXiv:1511.06581*,

2016. [Online]. Available: <https://arxiv.org/abs/1511.06581>.

[20] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 1998.

[21] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized Experience Replay," *arXiv preprint arXiv:1511.05952*, 2016. [Online]. Available: <https://arxiv.org/abs/1511.05952>.

[22] M. Samvelyan, T. Rashid, C. Schroeder de Witt, G. Farquhar, N. Nardelli, T. G. J. Rudner, C.-M. Hung, P. H. S. Torr, J. Foerster, and S. Whiteson, "The StarCraft Multi-Agent Challenge," *arXiv preprint arXiv:1902.04043*, 2019. [Online]. Available: <https://arxiv.org/abs/1902.04043>.

[23] W. Böhmer, V. Kurin, and S. Whiteson, "Deep Coordination Graphs," *arXiv preprint arXiv:1910.00091*, 2019. [Online]. Available: <https://doi.org/10.48550/arxiv.1910.00091>.

[24] K. De Asis, J. Hernandez-Garcia, G. Holland, and R. Sutton, "Multi-step Reinforcement Learning: A Unifying Algorithm," *arXiv preprint arXiv:1703.01327*, 2017. [Online]. Available: <https://doi.org/10.48550/arxiv.1703.01327>.

[25] K. De Asis and R. S. Sutton, "Per-decision Multi-step Temporal Difference Learning with Control Variates," *arXiv preprint arXiv:1807.01830*, 2018. [Online]. Available: <https://doi.org/10.48550/arxiv.1807.01830>.

9 Appendices

Hyperparameter	value	Hyperparameter	value
capture_action	Ture	n_stags	2
n_agents	4	n_haer	2
miscapture_punishment	0	agent_obs	[3, 3]
agent_move_block	[0, 1, 2]	capture_condition	[0, 1]
capture_action_conditions	[2, 1]	capture_freezes	True
capture_terminal	False	directed_observations	False
directed_cone_narrow	True	directed_exta_actions	True
episode_limit	300	intersection_global_view	False
intersection_unknown	False	mountain_slope	0.0
mountain_spawn	False	mountain_agent_row	-1
observe_state	False	observe_walls	False
observe_ids	False	observe_one_hot	False
p_stags_rest	0.1	p_hare_rest	0.5
prevent_cannibalism	True	print_caught_preay	False
print_frozen_agents	False	random_ghosts	False
random_ghosts_prob	0.5	random_ghosts_mul	-1
random_ghosts_indicator	False	remove_frozen	Frue
reward_hare	1	reward_stag	10

reward_collision	0	reward_time	-0.1
state_as_graph	False	toroidal	False
world_shape	[10, 10]	t_max	2000000
test_nepisode	16	test_interval	10000

Table 1: Hyperparameters for the baseline Predator-Prey environment